The Relay: an evidence-first RAG assistant over three papers

1 What I built and why

A Python assistant that answers only from three pinned arXiv papers (Attention v7, Moshi v1, Dynamic Chunking v2). Every answer carries a paper, a page, a section and a verbatim quote; the quote is checked by a tool call against the cited chunk *before* the answer is shown, and the system refuses rather than guess. Pipeline: analyse the question (is the corpus enough, which passages) -> BM25 retrieval over headings plus body (349 chunks, 350 tokens, 60 overlap) -> draft with JSON citations -> `verify_quote` -> at most two repairs -> refuse.

The decision that shaped everything else: **admission control on tokens, not retries on 429.** The free tier allows 30 requests per minute but only 8,000 tokens per minute. At roughly 1,200 tokens per answer the token ceiling binds about ten times earlier than the request ceiling, so the scheduler estimates cost before sending and refuses with a retry hint when a request cannot finish inside its deadline. Retries exist, but only for genuine failures.

2 The real service and my stand-in

Documented free-tier limits for `openai/gpt-oss-120b` (console.groq.com/docs/rate-limits, read 9 Nov 2026): 30 RPM, 1,000 RPD, 8,000 TPM, 200,000 TPD, 429 with `retry-after`. `relay probe-limits` is written to measure the observed side - 32 concurrent minimal calls for RPM, two 4.5k-token prompts for TPM, header capture, recovery polling, TTFT distribution - but **no key was in `.env` when I submitted**, so Table 1's observed column says *not measured*. I did not fill it with documentation dressed up as measurement.

The stand-in (`services/standin`) speaks the same OpenAI SSE protocol with real counters for those four limits, so the client code is identical for both sources. Its latency is fitted from real calls by `relay calibrate`; where it cannot match is listed in `params.json`: no cold start, TTFT independent of prefill length, provider queue depth approximated by a linear slowdown. Every row in the CSV carries `source`, and every load number in this report is labelled stand-in.

Table 1 - limits: documented versus observed

Limit	Documented	Observed	Evidence
Requests/min	30	not measured	probe needs `GROQ_API_KEY`
Requests/day	1,000	not measured	header value only, never exhausted
Tokens/min	8,000	not measured	header value only
Tokens/day	200,000	not measured	docs only, would cost 200k tokens
429 body + retry-after	429 + hint	not measured	probe needs `GROQ_API_KEY`
TTFT, small prompt	-	not measured	probe needs `GROQ_API_KEY`

3 Results

183 requests: 60 evaluation (20 questions x 3 settings) and 123 load, 53k tokens in and 3.8k out, 24 retries. Tables 2a and 2b come from `results/results.csv` via `relay tables`.

Table 2a - latency by scenario (ms)

Scenario	Req	TTFT p50	TTFT p95	TTFT p99	Total p50	Total p95	Total p99
steady	14	192	381	420	1,494	2,099	2,195
burst	30	140	1,598	2,159	7	13,254	13,982
overload	45	231	347	358	9	11,542	12,846
limit halved	22	225	751	903	3,856	39,358	48,115
daily exhausted	12	not measured	not measured	not measured	0	0	0

Table 2b - reliability, retries and cost

Scenario	Succeeded	Rate-limited	Timed out or given up	Retries/req	Peak queue	Tokens out/answer	Cost/1,000
steady	100%	0%	0%	0.00	1	75	$0.00
burst	3%	97%	0%	0.27	11	78	$0.00
overload	7%	93%	0%	0.18	8	82	$0.00
limit halved	45%	36%	18%	0.36	4	67	$0.00
daily exhausted	0%	100%	0%	0.00	1	not measured	$0.00

The number I could not have guessed: **30 requests per minute is not 30 answers per minute.** Offered at 4/min, everything succeeds (14/14, total p50 1.5 s). Offered 30 in three seconds, 3% succeed and the median response is 7 ms, because the token window is exhausted and the system refuses immediately instead of queueing. The advertised ceiling and the usable ceiling differ by a factor of ten, and the difference is entirely tokens.

4 Where the time goes

In steady state the median request takes 1,494 ms: 192 ms to the first token (13%) and 1,399 ms after it (87%). Working time is 1,594 ms, queue wait 0 ms, retry time 0 ms. Under burst, retrying time rises to 8 ms p50 and 24 retries appear, while queue wait stays at 0 - refusals happen before a request ever occupies a slot.

The split is structural, not accidental: the first model call decides answerability and is never shown to the user, so the token a person sees is the *second* call's first token. Everything after it is streaming ~75 output tokens, verifying the quote against the chunk, and occasionally one repair. Verification is not overhead to be optimised away; it is the reason the answer can be trusted, so the honest latency target is "first token early, verified answer 1.5 s later".

5 When a limit is hit

Per-minute: the provider returns 429 with `retry-after`. If that wait fits the request's deadline the limiter sleeps and retries (at most twice); otherwise it refuses at once with the wait in the message. Halving both windows gives the only scenario where waiting was rational: 45% succeeded, 36% were refused, 18% waited and timed out at p95 39.4 s.

Per-day: no wait inside a request can restore a daily budget, so the limiter rejects before sending - 0 retries, 0 ms, 100% `rate_limited`.

Who is delayed: only 18% of requests, and only in the halved scenario. Who is dropped: 97% in burst, 93% in overload, 100% when the day is gone. Who is degraded: the pipeline can drop to two passages and skip the analysis call under pressure, but in these runs refusals arrived first, so degradation was never exercised - I am not claiming a benefit I did not measure.

Retrieval: quality versus cost

Table 3 - retrieval settings (stand-in source, automatic grades)

Setting	Q	Correct+cited	Wrong	Refused correctly	Refused when answerable	Tokens out/answer	TTFT p50	Total p50
A: 350/60, k=4	20	25%	75%	0%	0%	81	167	1,680
B: 700/100, k=4	20	20%	80%	0%	0%	80	168	1,647
C: A + rerank 16->4	20	25%	75%	0%	0%	81	221	1,723

Three findings, one of them uncomfortable. Larger chunks (B) buy nothing: same tokens per answer, one fewer correct citation. Re-ranking (C) costs 54 ms of p50 time-to-first-token and 233 ms at p95 for the same accuracy. And the 0% refusal rate is a property of the *stand-in*, which always quotes its top

passage - a real model is what turns the refusal branch into behaviour, so Table 3 cannot choose a setting on quality. It can choose on cost, and both B and C cost more than A for no measured gain. The harness is ready: one command reruns all three against the real model.

How this library will mislead a naive retriever

1. **Appendix gravity.** Moshi is 66 pages and supplies 172 of 349 chunks; appendices repeat "LibriSpeech" and "WER" densely. `q06` (streaming WER, section 5.7) returns appendix tables first - the citation looks right and points at the wrong section.
2. **Cross-paper term collision.** `q11` asks for batch size and training steps (Attention section 5.1); the top four fill with Moshi and H-Net training sections that use exactly those words. This is why gold is a page-and-section region, not a chunk id.
3. **Heading versus body.** `q01` asks for the key dimension in section 3.2, but the body repeats "attention" and "heads" everywhere, so a body-only index returns generic 3.2.1 text. Indexing headings with each chunk is what lifted that passage into the top four; the miss was visible before the change and is recorded in the plan.

6 Retries

Full-jitter exponential backoff (base 400 ms, cap 8 s), three attempts per call, a retry budget shared across the run, and a hard rule that the delay is never shorter than the provider's `retry-after`. The circuit breaker counts only 5xx and timeouts: a 429 means the provider is healthy and busy.

Measured: 24 retries across 183 requests (0.13 per request), 0.27 in burst, 0.36 when the limits are halved. Of 21 retried requests, 3 succeeded, 17 ended rate-limited and 1 timed out. Retrying bought almost nothing, which is the point - with a 60-second window and a 30-second deadline, a retry is usually a slower way to refuse.

That rule came from a failure, not a principle: in the first full evaluation the breaker counted 429s as failures, opened for 20 seconds and turned 13 requests into instant `error` rows. The fix is decision D6 in `docs/DECISIONS.md`.

7 What the person sees

When the daily budget is gone, this is the reply, verbatim:

> *I can't answer this one right now: today's free-tier model budget is spent. The account allows 200,000 tokens and 1,000 requests per day, shared with every other run, and it resets at the start of the next UTC day. No partial or guessed answer was sent. Retry after the reset, or narrow the question so it needs fewer passages.*

At the per-minute limit the message names the wait ("It frees up in about 55 seconds. Nothing was lost - retry then.") and ends with the same promise: no shorter, unverified answer was sent to stay fast.

8 What I am unsure about

Least certain assumption: that `retry-after` is a good estimate of when capacity returns. It is a hint from one window; if it underestimates, the limiter sleeps, retries and fails anyway, which is strictly worse than refusing. What breaks first: multi-paper synthesis - nothing forces a cross-paper answer to cite more than one paper, and bucket C is where automatic grades flatter the system most.

Two things I did not build: multi-provider failover, and an answer cache keyed on normalised questions. Both would help at the margin; neither changes the token ceiling that dominates these results. One thing learned in my own words: a latency budget is set by the slowest *mandatory* step, and here that step is verification, so the second and a half after the first token is the product rather than the overhead.

Your feedback on Vaani

Standout: "zero dead air" and call screening that stays on the line through the carrier prompt are framed as outcomes, not model features - that is the right way to sell a voice agent, and the screening detail is the kind of unglamorous work that decides whether outbound campaigns survive.

Confusing: the landing page leads with p90 latency and "0.0% accuracy" style counters that render without denominators, and "start free" does not say what free includes. The first two questions a developer has - how fast, on what conditions; and what does it cost - are answered one click away instead of on the page.

One change: publish the measurement method next to the latency claim (audio conditions, line type, sample size), and put a three-line curl example beside it. For a latency-led product, the method is part of the pitch.

How I worked with the assistant

Three prompts that mattered: "plan before coding, with a gate per stage"; "measure the limits before designing the limiter"; and "every table must regenerate from `results.csv`". One thing it got wrong: it graded evaluation rows after the pipeline had already written them, producing duplicate, partly ungraded rows - caught by counting rows against questions. One overrule: it proposed a vector store for 349 chunks; BM25 over headings stayed. What is distinctly mine: the token-admission decision, the verify-before-reveal rule, the choice to label every stand-in row, and this report's refusal to fill unmeasured cells.